

Mais SQL, Álgebra e Plano de Consulta

Having

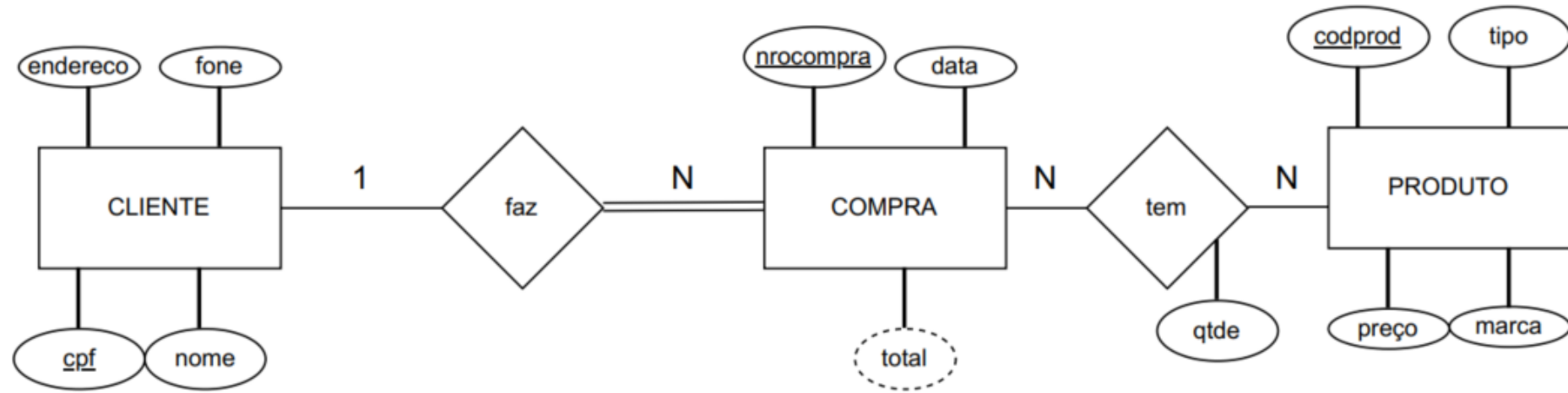
- No SQL, a cláusula HAVING é usada para filtrar resultados após um agrupamento (GROUP BY).

Selecione os tipos (categorias de produtos) que tem pelo menos 3 produtos e o número de produtos de cada tipo (categoria de produto).

```
SELECT tipo, COUNT(*) as total  
FROM Produto  
GROUP BY tipo  
HAVING COUNT(*) > 3;
```

Having

- A álgebra relacional **não tem uma operação chamada HAVING**, mas ela **trata essa lógica de outra forma**:
- Primeiro, faz o agrupamento (com agregação).
- Depois, filtra os resultados agregados usando a **seleção** (σ).
- Em outras palavras, usamos **seleção σ** sobre o resultado de um **agrupamento F** .



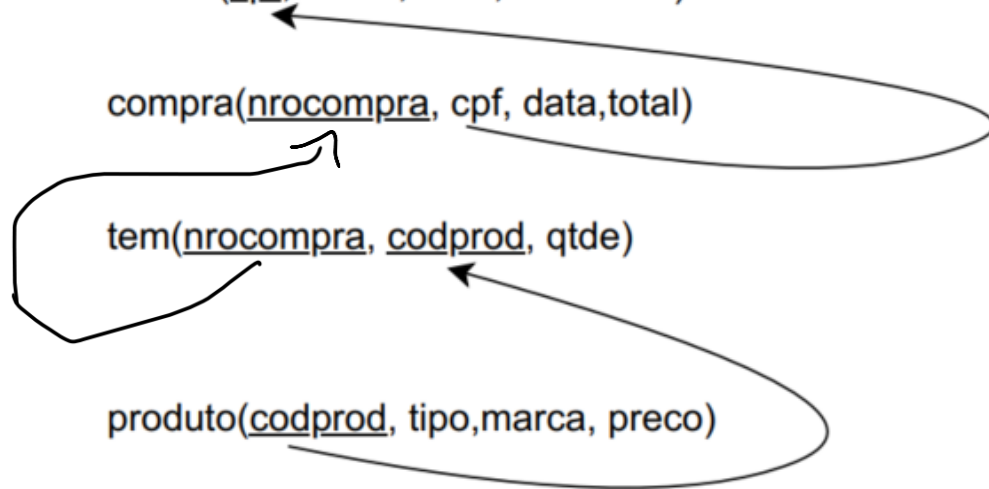
Mapamento para modelo Relacional

cliente(cpf, nome, fone, endereco)

compra(nrocompra, cpf, data, total)

tem(nrocompra, codprod, qtde)

produto(codprod, tipo, marca, preço)



```
SELECT tipo, COUNT(codprod) as total  
FROM Produto  
GROUP BY tipo  
HAVING COUNT(*) > 3;
```

$\sigma_{\text{count(codprod)} > 3} (\text{tipo } F_{\text{count(codprod)}} \text{ Produto})$

Estudo de Caso 1

- Consulta
- SQL
- Álgebra
- Plano
- Explain Analyse

- Consulta. Para cada produto, selecione o número de clientes que o comprou e o valor total comprado do produto.

- Consulta. Para cada produto, selecione o número de clientes que o comprou e o valor total comprado do produto.

```
SELECT p.codprod, count(c.cpf), sum(t.qtde) as qtde_vendida,  
sum(t.qtde)*(p.preco)  
FROM Produto p, Tem t, Compra c  
WHERE t.codprod = p.codprod and          SQL  
      c.nrocompra = t.nrocompra  
GROUP BY (p.codprod)  
ORDER BY p.codprod;
```

Inserindo na base para testar a resposta

```
INSERT INTO Tem (codprod, nrocompra, qtde) VALUES  
(1, 2, 2),  
(1, 3, 3),  
(1, 4, 4),  
(1, 5, 5);
```

- Consulta. Para cada produto, selecione o número de clientes que o comprou e o valor total comprado do produto.

```
SELECT p.codprod, count(c.cpf), sum(t.qtde) as qtde_vendida, sum(t.qtde)*(p.preco)
FROM Produto p, Tem t, Compra c
WHERE t.codprod = p.codprod and
      c.nrocompra = t.nrocompra
GROUP BY (p.codprod)
ORDER BY p.codprod;
```

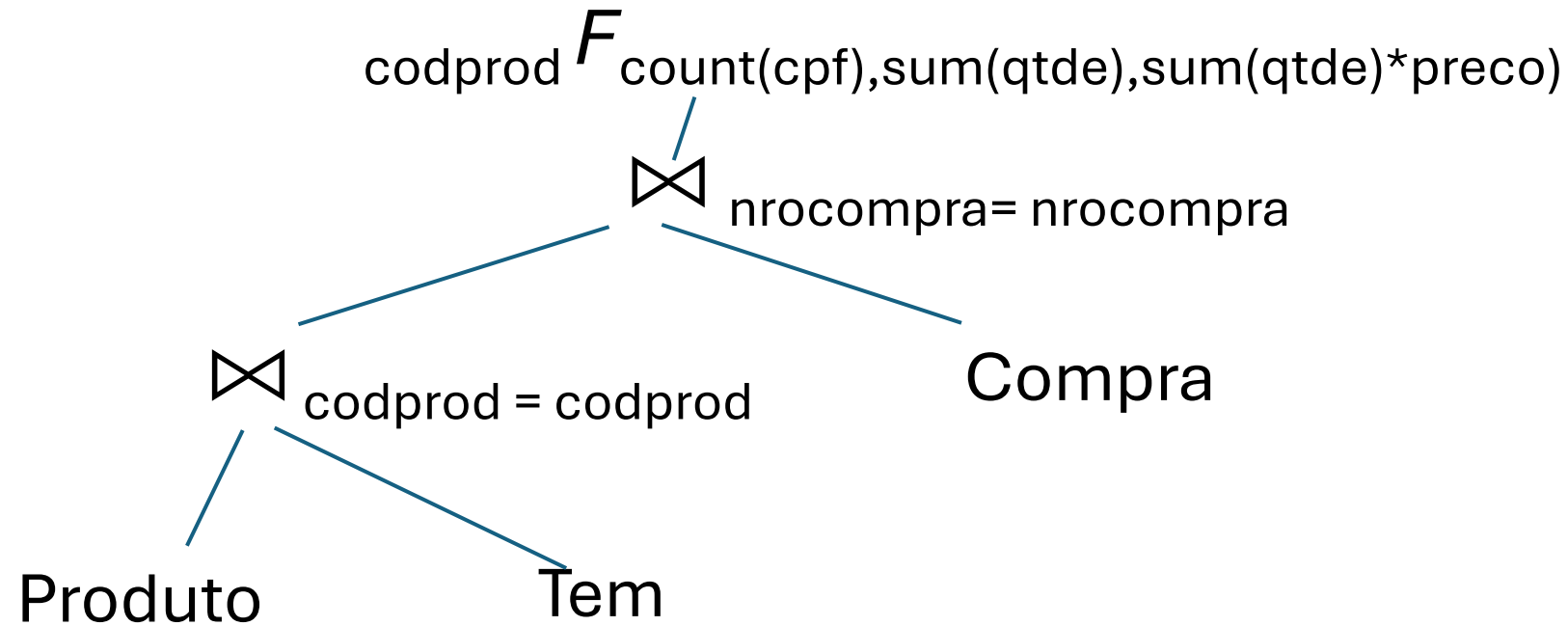
sql

$\bowtie$ codprod = codprod $\bowtie$ nrocompra =
 codprod F count(cpf), sum(qtde), sum(qtde)*preco) (Produto
 Tem
 Compra)
 nrocompra

Álgebra
relacional

Árvore de Consulta

$$\text{codprod } F_{\text{count}(\text{cpf}), \text{sum}(\text{qtde}), \text{sum}(\text{qtde}) * \text{preco}} (\text{Produto} \bowtie_{\text{codprod} = \text{codprod}} \text{Tem} \bowtie_{\text{nrocompra} = \text{nrocompra}} \text{Compra})$$



Explain Analyse

- Planos de Consulta: Uma Estrutura em Árvore
- Como Ler?
- Um plano de consulta pode ser pensado como uma estrutura em árvore, onde cada nó representa uma operação realizada pelo PostgreSQL.
- O plano descreve cada passo que o banco de dados realiza para completar uma consulta, começando pelas operações mais internas (nós folha) e avançando até o resultado final (nó raiz).

Lendo de Dentro para Fora

- Comece pelos nós mais internos e seguir para fora.
- Cada nó representa uma etapa do processo, e os nós mais externos normalmente agregam ou unem resultados dos nós internos.
- Cada nó geralmente depende dos dados processados pelos nós “abaixo” dele.
- Nós internos na Árvore: operações como junções, agregações ou limites.
- Nó Raiz: Este é o nó mais externo, representando a etapa final do processamento da consulta.

Custo — Estimativa do PostgreSQL de Recursos Computacionais

Cada nó inclui métricas de **estimativas de custo** e de **tempo real**, mostrando as estimativas do PostgreSQL para os recursos necessários. O custo é dividido em duas partes:

- **Startup Cost/Time (Custo Inicial):** para iniciar o nó.
- **Total Cost/Time (Custo Total):** para completar a tarefa do nó.
- **Plan Rows:** Número de linhas que o PostgreSQL espera processar.
- **Actual Rows:** Número real de linhas processadas durante a execução.

Query Query History

```
1  EXPLAIN ANALYSE
2  SELECT p.codprod, count(c.cpf), sum(t.qtde) as qtde_vendida, sum(t.qtde)*(p.preco)
3  FROM Produto p, Tem t, Compra c
4  WHERE t.codprod = p.codprod and
5         c.nrocompra = t.nrocompra
6  GROUP BY (p.codprod)
7  ORDER BY p.codprod;
```

Data Output Messages Graph Visualiser X Notifications



	QUERY PLAN	
	text	
10	Hash Cond: (t.codprod = p.codprod)	
11	-> Seq Scan on tem t (cost=0.00..30.40 rows=2040 width=12) (actual time=0.025..0.027 rows=...	
12	-> Hash (cost=12.90..12.90 rows=290 width=12) (actual time=0.027..0.027 rows=20 loops=1)	
13	Buckets: 1024 Batches: 1 Memory Usage: 9kB	
14	-> Seq Scan on produto p (cost=0.00..12.90 rows=290 width=12) (actual time=0.011..0.015 ...	
15	-> Hash (cost=18.50..18.50 rows=850 width=52) (actual time=0.020..0.020 rows=20 loops=1)	
16	Buckets: 1024 Batches: 1 Memory Usage: 9kB	

[Query](#) [Query History](#)

```
1  EXPLAIN ANALYSE
2  SELECT p.codprod, count(c.cpf), sum(t.qtde) as qtde_vendida, sum(t.qtde)*(p.p
3  FROM Produto p JOIN Tem t ON p.codprod = t.codprod
4           JOIN Compra c ON t.nrocompa = c.nrocompa
5  GROUP BY (p.codprod)
6  ORDER BY p.codprod;
```

[Data Output](#) [Messages](#) [Graph Visualiser](#) [X](#) [Notifications](#)

	QUERY PLAN	
	text	
5	Group Key: p.codprod	
6	Batches: 1 Memory Usage: 37kB	
7	-> Hash Join (cost=45.65..86.86 rows=2040 width=64) (actual time=0.060..0.072 rows=24 loops=1)	
8	Hash Cond: (t.nrocompa = c.nrocompa)	
9	-> Hash Join (cost=16.52..52.36 rows=2040 width=20) (actual time=0.041..0.048 rows=24 loops=1)	
10	Hash Cond: (t.codprod = p.codprod)	
11	-> Seq Scan on tem t (cost=0.00..30.40 rows=2040 width=12) (actual time=0.010..0.012 rows=...	
12	-> Hash (cost=12.90..12.90 rows=290 width=12) (actual time=0.015..0.015 rows=20 loops=1)	
13	Buckets: 1024 Batches: 1 Memory Usage: 9kB	
14	-> Seq Scan on produto p (cost=0.00..12.90 rows=290 width=12) (actual time=0.007..0.009 ...	
15	-> Hash (cost=18.50..18.50 rows=850 width=52) (actual time=0.013..0.013 rows=20 loops=1)	
16	Buckets: 1024 Batches: 1 Memory Usage: 9kB	
17	-> Seq Scan on compra c (cost=0.00..18.50 rows=850 width=52) (actual time=0.006..0.009 row...	
18	Planning Time: 0.251 ms	
19	Execution Time: 0.147 ms	

```
1  EXPLAIN ANALYSE
2  SELECT p.codprod, count(c.cpf), sum(t.qtde) as qtde_vendida, sum(t.qtde)*(p.preco)
3  FROM  Compra c, Produto p, Tem t
4  WHERE c.nrocompra = t.nrocompra and
5         t.codprod = p.codprod
6  GROUP BY (p.codprod)
7  ORDER BY p.codprod;
```

Data Output Messages Graph Visualiser X Notifications

	QUERY PLAN
	text
5	Group Key: p.codprod
6	Batches: 1 Memory Usage: 37kB
7	-> Hash Join (cost=45.65..86.86 rows=2040 width=64) (actual time=0.072..0.091 rows=24 loops=1)
8	Hash Cond: (t.codprod = p.codprod)
9	-> Hash Join (cost=29.12..64.90 rows=2040 width=56) (actual time=0.036..0.048 rows=24 loops=1)
10	Hash Cond: (t.nrocompra = c.nrocompra)
11	-> Seq Scan on tem t (cost=0.00..30.40 rows=2040 width=12) (actual time=0.006..0.009 rows=...
12	-> Hash (cost=18.50..18.50 rows=850 width=52) (actual time=0.015..0.015 rows=20 loops=1)
13	Buckets: 1024 Batches: 1 Memory Usage: 9kB
14	-> Seq Scan on compra c (cost=0.00..18.50 rows=850 width=52) (actual time=0.006..0.009 ...
15	-> Hash (cost=12.90..12.90 rows=290 width=12) (actual time=0.026..0.026 rows=20 loops=1)
16	Buckets: 1024 Batches: 1 Memory Usage: 9kB
17	-> Seq Scan on produto p (cost=0.00..12.90 rows=290 width=12) (actual time=0.013..0.017 row...
18	Planning Time: 0.262 ms
19	Execution Time: 0.187 ms

```
1  EXPLAIN ANALYSE
2  SELECT p.codprod, count(c.cpf), sum(t.qtde) as qtde_vendida, sum(t.qtde)*(p.preco)
3  FROM Compra c JOIN tem t ON t.nrocompra = c.nrocompra
4           JOIN Produto p ON p.codprod = t.codprod
5
6  GROUP BY (p.codprod)
7  ORDER BY p.codprod;
```

Data Output Messages Graph Visualiser X Notifications



	QUERY PLAN	
	text	
5	Group Key: p.codprod	
6	Batches: 1 Memory Usage: 37kB	
7	-> Hash Join (cost=45.65..86.86 rows=2040 width=64) (actual time=0.061..0.074 rows=24 loops=1)	
8	Hash Cond: (t.codprod = p.codprod)	
9	-> Hash Join (cost=29.12..64.90 rows=2040 width=56) (actual time=0.024..0.032 rows=24 loops=1)	
10	Hash Cond: (t.nrocompra = c.nrocompra)	
11	-> Seq Scan on tem t (cost=0.00..30.40 rows=2040 width=12) (actual time=0.005..0.007 rows=...	
12	-> Hash (cost=18.50..18.50 rows=850 width=52) (actual time=0.011..0.012 rows=20 loops=1)	
13	Buckets: 1024 Batches: 1 Memory Usage: 9kB	
14	-> Seq Scan on compra c (cost=0.00..18.50 rows=850 width=52) (actual time=0.005..0.007 ...	
15	-> Hash (cost=12.90..12.90 rows=290 width=12) (actual time=0.023..0.023 rows=20 loops=1)	
16	Buckets: 1024 Batches: 1 Memory Usage: 9kB	
17	-> Seq Scan on produto p (cost=0.00..12.90 rows=290 width=12) (actual time=0.012..0.015 row...	
18	Planning Time: 0.206 ms	
19	Execution Time: 0.151 ms	

```
1  ✓ EXPLAIN ANALYSE
2  SELECT p.codprod, count(c.cpf), sum(t.qtde) as qtde_vendida, sum(t.qtde)*(p.preco)
3  FROM  Compra c NATURAL JOIN Tem t
4          NATURAL JOIN PRODUTO p
5  GROUP BY (p.codprod)
6  ORDER BY p.codprod;
```

Data Output Messages Graph Visualiser X Notifications



	QUERY PLAN	
	text	🔒
5	Group Key: p.codprod	
6	Batches: 1 Memory Usage: 37kB	
7	-> Hash Join (cost=45.65..86.86 rows=2040 width=64) (actual time=0.070..0.080 rows=24 loops=1)	
8	Hash Cond: (t.codprod = p.codprod)	
9	-> Hash Join (cost=29.12..64.90 rows=2040 width=56) (actual time=0.024..0.029 rows=24 loops=1)	
10	Hash Cond: (t.nrocompra = c.nrocompra)	
11	-> Seq Scan on tem t (cost=0.00..30.40 rows=2040 width=12) (actual time=0.006..0.008 rows=...	
12	-> Hash (cost=18.50..18.50 rows=850 width=52) (actual time=0.010..0.010 rows=20 loops=1)	
13	Buckets: 1024 Batches: 1 Memory Usage: 9kB	
14	-> Seq Scan on compra c (cost=0.00..18.50 rows=850 width=52) (actual time=0.005..0.006 ...	
15	-> Hash (cost=12.90..12.90 rows=290 width=12) (actual time=0.036..0.037 rows=20 loops=1)	
16	Buckets: 1024 Batches: 1 Memory Usage: 9kB	
17	-> Seq Scan on produto p (cost=0.00..12.90 rows=290 width=12) (actual time=0.026..0.028 row...	
18	Planning Time: 0.368 ms	
19	Execution Time: 0.184 ms	

O que a consulta a seguir faz?

```
SELECT p.codprod, count(c.cpf), sum(t.qtde) as qtde_vendida,  
sum(t.qtde)*(p.preco)  
FROM Compra c, Produto p, Tem t  
WHERE p.codprod = t.codprod and  
      t.nrocompra = c.nrocompra  
GROUP BY (p.codprod)  
HAVING count(distinct c.nrocompra) > 1  
ORDER BY p.codprod
```